

Unidade III

Nesta unidade, o percurso acadêmico evolui da estruturação de interfaces para a engenharia de aplicações dinâmicas e orientadas a dados. Se nas etapas anteriores o foco residiu na ergonomia visual e na reatividade da UI, nesta fase o desenvolvimento alcança a camada de infraestrutura, onde o aplicativo atua como um nó em um ecossistema distribuído de serviços.

Aplicações móveis contemporâneas operam sob o paradigma cliente-servidor, dependendo de uma comunicação robusta com APIs para garantir a persistência e a integridade das informações. O domínio técnico desta unidade capacitará o aluno a transformar interfaces estáticas em sistemas funcionais capazes de processar informações complexas, gerenciar estados globais e utilizar o hardware do dispositivo de forma estratégica.

Em conformidade com as exigências de densidade conceitual, serão abordados:

- **Comunicação com APIs RESTful:** compreensão de contratos de interface, verbos HTTP (GET, POST, PUT, DELETE) e arquitetura cliente-servidor.
- **Processamento de dados (JSON):** serialização e desserialização tipada de dados, utilizando DTOs (Data Transfer Objects) para garantir a segurança da informação.
- **Persistência e sincronização local:** implementação de bancos de dados locais (SQLite) e estratégias de cache para funcionamento offline-first.
- **Utilização de recursos nativos:** integração com sensores de hardware (GPS, câmera) e tratamento de permissões em tempo de execução.
- **Segurança e confiabilidade:** implementação de HTTPS, autenticação via tokens e tratamento de exceções por camadas.

Assim, abordaremos uma sequência de implementação guiada, onde cada conceito técnico será acompanhado de blocos de código comentados e justificativas arquiteturais, permitindo a construção de soluções escaláveis e alinhadas às expectativas do mercado global de tecnologia.

5 INTEGRAÇÃO DE DADOS E FUNCIONALIDADES AVANÇADAS

5.1 Consumo de dados e arquitetura de integração com APIs

No ecossistema de desenvolvimento móvel, as APIs (application programming interfaces) atuam como a camada de abstração que permite a interoperabilidade entre o dispositivo cliente e servidores remotos. No contexto contemporâneo, a quase totalidade das aplicações de alto impacto – de sistemas bancários a plataformas de streaming – opera sob o paradigma de sistemas distribuídos, onde a inteligência de processamento e o armazenamento de grandes volumes de dados residem na nuvem.

Segundo Fielding (2000), o estilo arquitetural REST (representational state transfer) estabelece um conjunto de restrições que garantem a escalabilidade e a visibilidade da comunicação. APIs baseadas em REST utilizam o protocolo HTTP de forma desacoplada, permitindo que a interface móvel evolua independentemente da lógica do servidor.

Durante a dinâmica de comunicação cliente-servidor, o fluxo de integração segue um ciclo de vida de requisição e resposta padronizado:

- **Requisição (request):** o aplicativo móvel (cliente) envia um pacote de dados via HTTP para um endpoint específico (como GET/clima/sao-paulo).
- **Processamento:** o servidor processa a solicitação, consulta bancos de dados e aplica regras de negócio.
- **Resposta (response):** o servidor devolve um código de status (por exemplo, 200 OK) acompanhado de um corpo de dados, geralmente em formato JSON.
- **Renderização:** o aplicativo interpreta os dados recebidos e atualiza a árvore de widgets de forma reativa.

Quanto a relevância estratégica das APIs, podemos notar:

- **Atualização dinâmica:** permitem a modificação de conteúdos em tempo real sem a necessidade de submeter uma nova versão do aplicativo às lojas (App Store/Google Play).
- **Segurança de dados:** centralizam a lógica sensível no servidor, reduzindo a superfície de ataque no dispositivo do usuário.
- **Economia de recursos:** delegam tarefas de processamento pesado para servidores robustos, preservando a bateria e o hardware do smartphone.

A arquitetura RESTful consolidou o formato JSON (JavaScript Object Notation) como o padrão ouro para troca de mensagens devido à sua leveza e facilidade de leitura por humanos e máquinas. Para o desenvolvedor de nível superior, é crucial compreender que a integração robusta exige o tratamento de exceções de rede, latência e a implementação de cache para garantir a disponibilidade do sistema.



Observação

Em conformidade com o rigor técnico, o consumo de APIs deve ser sempre acompanhado de protocolos de segurança, como o HTTPS, e autenticação via tokens (como JWT), garantindo que apenas usuários autorizados acessem recursos sensíveis do sistema.

5.2 Arquitetura cliente-servidor e o papel estratégico das APIs

No desenvolvimento de sistemas móveis de larga escala, a adoção da arquitetura cliente-servidor é imperativa para garantir a integridade e a persistência dos dados. Neste modelo, o processamento é distribuído entre dois componentes principais: o **cliente** (a aplicação instalada no dispositivo móvel), responsável pela lógica de apresentação e interação; e o **servidor** (composto por APIs, serviços de nuvem e bancos de dados), responsável pela persistência, regras de negócio complexas e segurança.

Segundo Sommerville (2016), as arquiteturas cliente-servidor são fundamentais para o desenvolvimento de sistemas distribuídos, pois favorecem a escalabilidade, a centralização da lógica e a facilidade de manutenção.

Diferente de uma visão simplista, a API não é apenas uma ponte, mas uma interface de contrato que define como dois sistemas se comunicam de forma segura e padronizada. A utilização de APIs RESTful permite que a aplicação mobile atue como um consumidor de recursos, delegando tarefas pesadas de hardware para infraestruturas robustas em nuvem. Essa abordagem arquitetural possibilita:

- **Sincronização multiplataforma:** garantia de que o usuário acesse os mesmos dados em diferentes dispositivos (Android, iOS ou Web) de forma agnóstica.
- **Autenticação centralizada:** implementação de protocolos de segurança como OAuth 2.0 e JWT para validar a identidade do usuário de forma segura e persistente.
- **Eficiência de manutenção:** alterações nas regras de negócio ou nos dados podem ser feitas diretamente no servidor, impactando instantaneamente todos os clientes sem a necessidade de novos builds nas lojas de aplicativos.



Saiba mais

As arquiteturas cliente-servidor evoluíram significativamente com o avanço da computação em nuvem e dos sistemas distribuídos. Atualmente, aplicações móveis podem utilizar APIs REST e GraphQL para a comunicação com serviços remotos, favorecendo a integração entre diferentes plataformas e arquiteturas.

Para aprofundar seus conhecimentos sobre GraphQL, recomenda-se a leitura da documentação oficial Learn, que apresenta conceitos fundamentais dessa linguagem de consulta para APIs, incluindo esquemas, tipos, consultas, mutações e execução de operações.

LEARN. *GraphQL*, [s.d.]. Disponível em: <https://tinyurl.com/3366hc6j>. Acesso em: 7 jul. 2026.

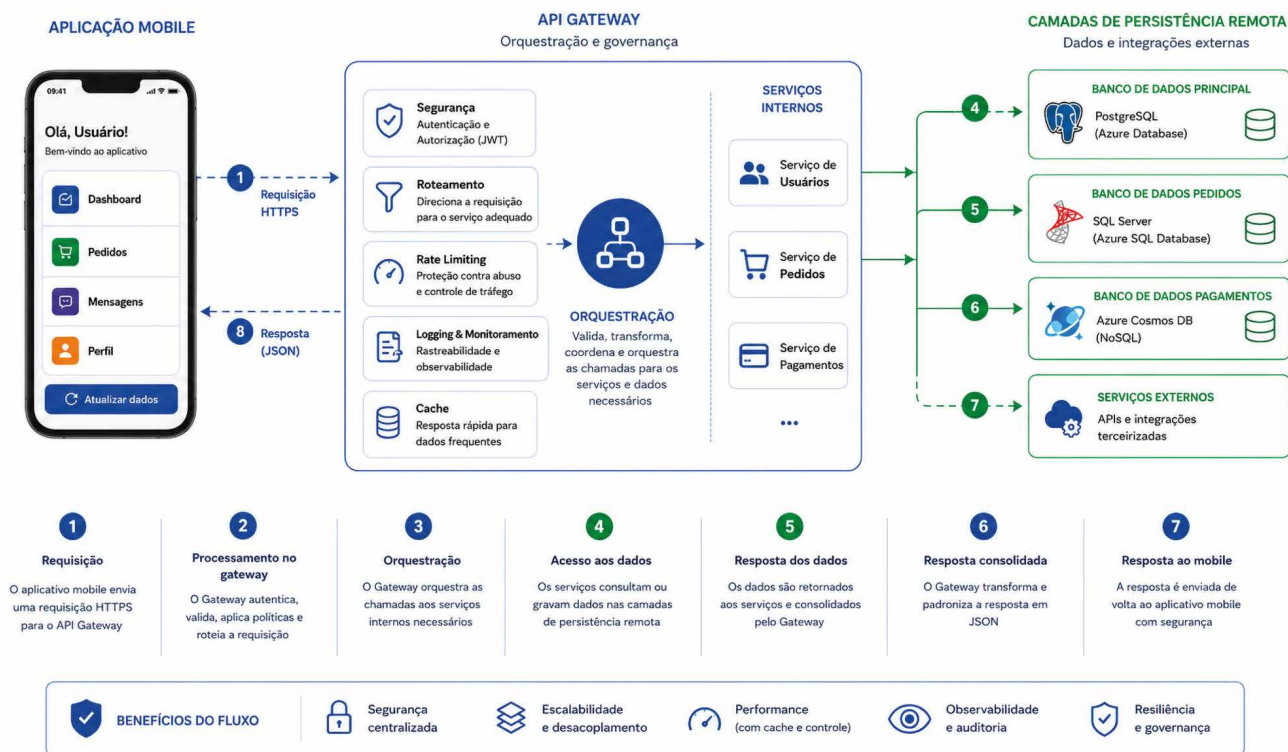


Figura 19 – Fluxo de comunicação e orquestração entre aplicação mobile, API Gateway e camadas de persistência remota (imagem produzida com a ferramenta de IA generativa ChatGPT)

As APIs permitem que o conteúdo das aplicações seja atualizado dinamicamente em tempo real. No entanto, para o desenvolvedor de nível superior, é crucial projetar sistemas resilientes que tratem cenários de latência de rede e indisponibilidade temporária do servidor, utilizando estratégias de cache local e bancos de dados – como o SQLite – para garantir a continuidade da experiência do usuário (offline-first).

Em arquiteturas profissionais, o consumo de APIs deve seguir o Repository Pattern, isolando a camada de dados da interface do usuário. Isso facilita a implementação de testes unitários e permite a troca de fontes de dados (por exemplo, mudar de uma API real para um mock de testes) de forma transparente para a aplicação.

5.3 Comunicação assíncrona e protocolo HTTP

As requisições HTTP (Hypertext Transfer Protocol) constituem o alicerce da comunicação entre aplicações móveis e serviços de backend. No desenvolvimento profissional, essa interação é inerentemente assíncrona, exigindo que o desenvolvedor gerencie estados de espera, sucesso e falha para manter a fluidez da interface e a integridade dos dados.

Métodos HTTP e semântica de operações

Os métodos HTTP possuem uma semântica específica de operação. Assim, a comunicação com APIs RESTful baseia-se em métodos que definem a natureza da operação no servidor (CRUD). O uso correto da semântica HTTP é fundamental para a escalabilidade e interoperabilidade do sistema:

- **GET (leitura):** solicita a representação de um recurso. Não deve causar efeitos colaterais no servidor.
- **POST (escrita):** submete dados para serem processados, resultando frequentemente na criação de um novo recurso.
- **PUT (atualização):** substitui todas as representações atuais do recurso de destino pelos dados da requisição.
- **DELETE (remoção):** exclui o recurso especificado no servidor.

Vejam os a seguir um exemplo de implementação de requisição em Dart que utiliza blocos de texto comentados para facilitar a análise técnica.

```
import 'package:http/http.dart' as http;
import 'dart:convert';

// Exemplo de busca de dados (GET) com tratamento de estado
Future<void> buscarPerfil(int id) async {
  try {
    final response = await http.get(
      Uri.parse('https://api.exemplo.com/usuarios/$id'),
      headers: {'Authorization': 'Bearer <token>'},
    ).timeout(Duration(seconds: 10)); // Controle de Timeout
    if (response.statusCode == 200) {
      var dados = jsonDecode(response.body);
      print('Usuário: ${dados['nome']}');
    } else {
      throw Exception('Erro ao carregar perfil: ${response.statusCode}');
    }
  }
}
```

```

} catch (e) {

    print('Falha na comunicação: $e'); // Tratamento de erros por camada

}

}

```

Ainda, toda transação HTTP retorna um código de status, que deve ser validado para garantir que a aplicação reaja corretamente a diferentes cenários de rede e lógica de backend.

Quadro 11 – Códigos de status e tratamento em mobile

Código	Classe	Significado técnico	Ação sugerida no app
200/201	Sucesso	Requisição processada com êxito	Atualizar UI com os dados recebidos
400	Erro do cliente	Requisição malformada ou erro de validação	Exibir alerta de erro ao usuário
401/403	Erro de permissão	Token ausente, expirado ou inválido	Redirecionar para tela de login (refresh token)
404	Não encontrado	O endpoint ou recurso não existe	Exibir tela de "Conteúdo não encontrado"
500	Erro do servidor	Falha interna no processamento do backend	Sugerir tentativa posterior ao usuário

Para mitigar vulnerabilidades críticas, como o armazenamento inseguro de credenciais – apontado pelo OWASP Foundation (s.d.) como uma das principais ameaças –, a integração com APIs deve seguir protocolos rigorosos:

- **Tráfego criptografado:** uso mandatório de HTTPS para prevenir ataques de Man-in-the-Middle.
- **Autenticação robusta:** implementação de JWT (JSON Web Token) com armazenamento em locais seguros do hardware (como EncryptedSharedPreferences no Android ou KeyChain no iOS).
- **Controle de latência:** implementação de timeouts e estratégias de retry automático para redes instáveis.
- **Serialização tipada:** uso de DTOs para evitar inconsistências entre o JSON recebido e os objetos de domínio do app.



Observação

Para Nielsen (1993), a interface deve fornecer feedback imediato (como shimmer effects ou spinners) durante o processamento de requisições assíncronas, evitando a percepção de travamento do sistema.

5.4 Manipulação e serialização de dados (JSON)

O JSON (JavaScript Object Notation) consolidou-se como o padrão global para o intercâmbio de dados em sistemas distribuídos devido à sua sintaxe leve, independente de linguagem e de fácil interpretação por algoritmos de parsing. No desenvolvimento mobile, o domínio da manipulação de JSON é o que permite a transição de interfaces estáticas para sistemas dinâmicos e alimentados por dados.

Estruturalmente, o JSON opera através de coleções de pares de chave e valor e listas ordenadas. Sua natureza flexível exige que o desenvolvedor implemente camadas de validação para garantir a integridade dos dados consumidos.

Quando um aplicativo consome uma API, os dados chegam como uma string bruta. O processo de tradução dessa string para estruturas de dados manipuláveis pelo Dart é conhecido como desserialização. No Flutter, a biblioteca nativa `dart:convert` é a responsável por essa tarefa.



Saiba mais

O formato JSON é amplamente utilizado para a troca de informações entre aplicações web, móveis e serviços em nuvem devido à sua simplicidade, leveza e independência em relação à linguagem de programação. Em aplicações desenvolvidas com Dart e Flutter, a biblioteca `dart:convert` oferece recursos nativos para codificação e decodificação de dados JSON. Em projetos de maior complexidade, ferramentas de geração de código, como `json_serializable`, podem auxiliar na automatização dos processos de serialização e desserialização de objetos, contribuindo para a redução de erros e para a manutenção do código.

Para aprofundar seus conhecimentos sobre o uso de JSON em Dart, recomenda-se a leitura da documentação a seguir, que apresenta estratégias para codificação e decodificação de dados, serialização de objetos e uso de soluções baseadas em geração de código.

USING JSON. *Dart*, 2 fev. 2026. Disponível em:
<https://tinyurl.com/mumhx9wy>. Acesso em: 7 jul. 2026.

O código a seguir apresenta a implementação dessa conversão e o acesso a dados.

```
import 'dart:convert';

void processarResposta(String responseBody) {

  // Converte a String JSON em um Map dinâmico

  final Map<String, dynamic> dados = jsonDecode(responseBody);
```

```
// Acesso seguro utilizando o operador Null-Coalescing (??)

final String titulo = dados['titulo'] ?? 'Título indisponível';

print('Processando: $titulo');

}
```

Modelagem de dados e DTOs (data transfer objects)

Para projetos de nível profissional, o parecer técnico veda o uso exclusivo de Maps genéricos (como `dados['campo']`), pois essa prática é propensa a erros de digitação e dificulta a manutenção. A boa prática de engenharia recomenda a criação de classes de modelo com construtores do tipo `factory`. Essa abordagem garante tipagem forte, permitindo que o compilador identifique erros antes mesmo da execução.

```
class Post {

    final int id;

    final String titulo;

    Post({required this.id, required this.titulo});

    // Factory constructor para converter JSON em objeto Post

    factory Post.fromJson(Map<String, dynamic> json) {

        return Post(

            id: json['id'] as int,

            titulo: json['titulo'] as String? ?? 'Sem título',

        );

    }

}
```

A manipulação de dados externos é uma operação de risco. Falhas de rede, JSONs malformados ou campos ausentes podem causar o encerramento inesperado (crash) da aplicação. Logo, o uso de blocos `try-catch` e verificações de tipo é mandatório.

Quadro 12 – Riscos e mitigações na manipulação de dados

Risco técnico	Consequência	Mitigação recomendada
Chave inexistente	Valor nulo/crash	Uso do operador <code>??</code> e <code>Null Safety</code>
Tipo incorreto	Exception de cast	Validação com o operador <code>is</code> (por exemplo, <code>dados is Map</code>)
JSON corrompido	Falha no <code>jsonDecode</code>	Enclausuramento em blocos <code>try-catch</code>



Observação

Em aplicações de larga escala, utilize bibliotecas de geração de código como `json_serializable`. Elas automatizam a criação dos métodos `fromJson`, reduzindo o erro humano e garantindo que o mapeamento de dados siga padrões rigorosos de segurança e performance.

5.5 Tratamento de exceções e resiliência em sistemas móveis

No desenvolvimento de aplicações distribuídas, o tratamento de erros transcende a simples captura de falhas; trata-se de projetar um sistema resiliente capaz de manter a estabilidade e a confiabilidade mesmo diante de instabilidades de infraestrutura. Aplicações móveis operam em ambientes altamente voláteis (redes 3G/4G/5G oscilantes, modo avião, esgotamento de bateria), o que torna a gestão de exceções um pilar crítico da engenharia de software.

Na arquitetura de tratamento por camadas, o uso do bloco `try-catch` em Dart permite que o desenvolvedor capture exceções de diferentes naturezas e responda a elas de forma estratégica, evitando o encerramento inesperado (crash) do processo.

```
import 'package:http/http.dart' as http;

import 'dart:async'; // Necessário para TimeoutException

Future<void> realizarIntegracaoSegura() async {

  try {

    final url = Uri.parse("https://api.exemplo.com/v1/posts/1");

    // Implementação de Timeout para evitar processos "zumbis"

    final response = await http.get(url).timeout(const Duration(seconds: 10));

    // Validação da Semântica HTTP

    if (response.statusCode == 200) {

      final dados = jsonDecode(response.body);

      // Processamento lógico...

    } else {

      // Tratamento de erros de negócio (4xx e 5xx)

      _logError("Erro de Servidor: ${response.statusCode}");

    }

  }

} on TimeoutException catch (_) {
```

```
        _notificarUsuario("A conexão expirou. Verifique sua rede.");  
    } on SocketException catch (e) {  
        _notificarUsuario("Sem conectividade com a internet.");  
    } catch (e) {  
        // Captura de exceções genéricas ou de parsing  
        _logError("Falha inesperada: $e");  
    }  
}
```

Para um desenvolvedor de nível superior, é fundamental distinguir as falhas por sua origem e aplicar a técnica de mitigação adequada:

- **Exceções de rede (SocketException):** causadas por ausência de sinal ou DNS inválido. A solução envolve o uso de bibliotecas de monitoramento de conectividade (Connectivity Plus).
- **Exceções de protocolo (Status 4xx/5xx):** o servidor respondeu, mas não pôde processar o pedido. Exige lógica de negócio para redirecionamento (por exemplo, 401 para login) ou mensagens de erro amigáveis.
- **Exceções de tempo (TimeoutException):** ocorrem quando o servidor demora a responder. É essencial para evitar o bloqueio da interface e o consumo excessivo de recursos.
- **Exceções de dados (FormatException):** ocorrem quando o JSON recebido está corrompido ou malformatado.



Lembrete

A comunicação entre aplicações móveis e servidores deve ocorrer por meio do protocolo HTTPS, garantindo a confidencialidade e a integridade dos dados transmitidos.

No que toca as boas práticas e os feedbacks a usuários, segundo Nielsen (1993), a visibilidade do status do sistema é um princípio de usabilidade. Portanto, o tratamento de erros deve ser acompanhado de componentes visuais:

- **Snackbar/Dialogs:** para erros não fatais e informativos.
- **Empty states:** telas personalizadas quando uma lista não pode ser carregada por erro de rede.
- **Retry pattern:** implementação de mecanismos de tentativa automática com atraso exponencial (exponential backoff).



Observação

Nunca exiba o rastro de pilha (stack trace) ou mensagens técnicas brutas da API diretamente para o usuário final. Isso pode expor detalhes da infraestrutura de backend para potenciais atacantes, além de prejudicar a experiência do usuário.

5.6 Exemplo prático integrado: consumo de endpoints REST

A integração com serviços externos exige uma orquestração precisa entre a requisição de rede, o processamento de dados e a gestão de exceções. O exemplo a seguir demonstra a implementação de um serviço de busca de usuários utilizando a API JSONPlaceholder.

Diferente de scripts simples, uma implementação robusta em nível de engenharia deve prever falhas de conectividade e garantir que o fluxo de dados respeite o contrato estabelecido pela API. O código a seguir traz um exemplo dessa implementação.

```
import 'package:http/http.dart' as http;

import 'dart:convert';

import 'dart:io';

/// Realiza o consumo de uma lista de usuários de forma assíncrona.

Future<void> carregarUsuarios() async {

  try {

    // Passo 1: Definição do Endpoint e conversão para Objeto Uri

    final url = Uri.parse('https://jsonplaceholder.typicode.com/users');

    // Passo 2: Execução da requisição GET com await

    final response = await http.get(url).timeout(const Duration(seconds: 15));

    // Passo 3: Validação da Semântica HTTP (Status 200 OK)

    if (response.statusCode == 200) {

      // Passo 4: Desserialização do corpo da resposta (String para List)

      final List<dynamic> dados = jsonDecode(response.body);

      // Passo 5: Validação de estrutura e acesso seguro (Null Safety)

      if (dados.isNotEmpty) {
```

```

        final String nomePrimeiroUsuario = dados[0]['name'] ?? 'Usuário sem
nome';

        print('Integração bem-sucedida: $nomePrimeiroUsuario');

    }

} else {

    print('Falha no servidor: Código ${response.statusCode}');

}

} on SocketException {

    print('Erro de rede: Certifique-se de que o dispositivo está conectado.');
```

```

} catch (e) {

    // Captura de exceções genéricas (ex: falha no parsing do JSON)

    print('Erro inesperado na integração: $e');
```

```

}

}

```

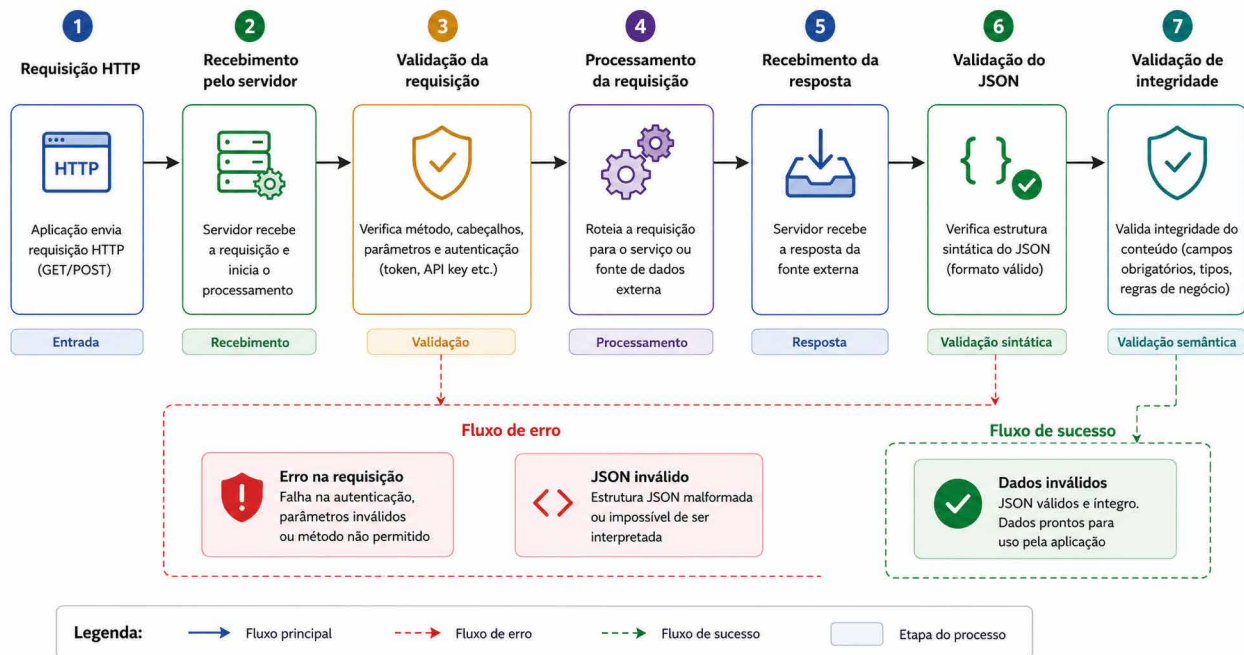


Figura 20 – Fluxo lógico de processamento de dados externos: da requisição HTTP à validação de integridade do JSON (imagem produzida com a ferramenta de IA generativa ChatGPT)



Lembrete

Ao consumir APIs REST, nunca assuma que a resposta será sempre válida. Antes de processar os dados, verifique o código de status HTTP, valide a estrutura do JSON recebido e trate situações como indisponibilidade do servidor, tempo limite de resposta (timeout) e perda de conexão. Essas verificações tornam a aplicação mais confiável e proporcionam uma experiência mais segura ao usuário.

Na análise da arquitetura, como apresentada na figura anterior, destacam-se os seguintes componentes críticos:

- **Contexto assíncrono (async/await):** essencial para operações de I/O (entrada e saída). O `await` suspende a execução da função sem bloquear a main thread, permitindo que a interface do usuário permaneça responsiva durante a espera pelo servidor.
- **Validação de tipo (is List):** antes de iterar ou acessar índices (como `dados[0]`), é imperativo validar se o JSON retornado é de fato uma coleção. APIs podem retornar objetos de erro em vez de listas em cenários de falha.
- **Gestão de timeouts:** a inclusão do método `.timeout()` evita que a aplicação aguarde indefinidamente por uma resposta de um servidor instável, o que drenaria recursos de processamento e bateria.
- **Operador Null-Coalescing (??):** utilizado para fornecer valores padrão caso uma chave esperada no JSON esteja ausente, prevenindo erros de execução em tempo de execução.



Observação

Embora este exemplo execute o processamento diretamente na função, em arquiteturas escaláveis (como clean architecture), esta lógica residiria em uma camada de data source, sendo exposta para a interface através de um repository, garantindo o desacoplamento total entre a lógica de rede e a visualização.

6 ARMAZENAMENTO E RECURSOS DO DISPOSITIVO

6.1 Estratégias de persistência de dados local

No desenvolvimento mobile, a capacidade de persistir dados localmente é um requisito funcional crítico para garantir a resiliência e a performance da aplicação. A persistência local permite que o software mitigue a latência de rede, suporte operações offline e armazene preferências de personalização que melhoram a experiência do usuário.

Dependendo da complexidade e da sensibilidade da informação, o desenvolvedor deve optar por diferentes mecanismos de armazenamento, variando desde arquivos de configuração simples até bancos de dados relacionais robustos.

Uma das estratégias a serem implementadas ocorre com a persistência de chave-valor com SharedPreferences. O SharedPreferences é uma API que permite o armazenamento de dados primitivos (strings, inteiros, booleanos) em um formato de mapa persistente. Tecnicamente, no Android, esses dados são gravados em arquivos XML privados, enquanto no iOS utilizam o UserDefaults. Veja o exemplo de código a seguir.

```
import 'package:shared_preferences/shared_preferences.dart';

/// Persiste as preferências de interface do usuário

Future<void> salvarConfiguracoes(String tema, bool notificacoes) async {

  // Obtém a instância do repositório local

  final SharedPreferences prefs = await SharedPreferences.getInstance();

  // Escrita assíncrona de dados primitivos

  await prefs.setString('tema_app', tema);

  await prefs.setBool('status_notificacoes', notificacoes);

}

/// Recupera dados persistidos com tratamento de nulidade

Future<void> carregarConfiguracoes() async {

  final prefs = await SharedPreferences.getInstance();

  // O uso do operador '?' indica que o valor pode ser nulo (Null Safety)

  final String? temaAtivo = prefs.getString('tema_app');

  // Definição de valor padrão caso a chave não exista (Null-coalescing)

  print('Tema recuperado: ${temaAtivo ?? "Padrão do Sistema"}');

}
```

Embora eficiente, o SharedPreferences não deve ser utilizado para:

- **Grandes volumes de dados:** listas extensas ou objetos complexos podem causar lentidão na leitura/escrita.
- **Dados relacionais:** para buscas complexas, deve-se utilizar o SQLite.
- **Informações sensíveis:** dados gravados via SharedPreferences não são criptografados nativamente.

Conforme a OWASP Foundation (s.d.), o armazenamento inseguro de segredos de autenticação é uma das principais vulnerabilidades em dispositivos móveis. Informações como tokens JWT, senhas ou dados bancários devem ser obrigatoriamente protegidos por camadas de criptografia de hardware.

No Flutter, utiliza-se o plugin `flutter_secure_storage`, que implementa:

- **Android:** armazenamento no KeyStore com criptografia AES.
- **iOS:** persistência no KeyChain, garantindo que os dados não sejam acessíveis mesmo em dispositivos com jailbreak.

Quadro 13 – Comparativo de persistência local

Tecnologia	Tipo de dado	Segurança nativa	Recomendação técnica
SharedPreferences	Primitivos (key-value)	Baixa (texto puro)	Preferências de UI e flags simples
Secure Storage	Strings criptografadas	Alta (hardware)	Tokens de API e credenciais
SQLite (sqflite)	Dados estruturados	Média (extensível)	Históricos, cache de API e dados complexos



Observação

Ao recuperar dados locais, sempre implemente a validação de nulidade. Como o armazenamento reside no hardware do usuário, o aplicativo deve estar preparado para cenários onde os arquivos de dados foram limpos pelo sistema operacional ou pelo próprio usuário nas configurações do dispositivo.

6.2 Persistência estruturada com SQLite

Enquanto o SharedPreferences é limitado a metadados simples, aplicações que gerenciam volumes expressivos de informação ou que exigem relacionamentos complexos utilizam o SQLite. Trata-se de um motor de banco de dados relacional leve, baseado em arquivos, que opera de forma embarcada (embedded) no dispositivo móvel.

O SQLite permite a execução de comandos SQL (Structured Query Language) padronizados, garantindo que o desenvolvedor possa realizar filtros, ordenações e relacionamentos entre tabelas (como Joins), essenciais para sistemas de gestão de tarefas, catálogos de produtos e históricos de transações.

Seguindo os princípios de Martin (2019) em clean architecture, o acesso ao banco de dados não deve ser feito diretamente na camada de interface. Para garantir a testabilidade e a manutenção, utilizamos dois padrões fundamentais:

- **DAO (Data Access Object)**: classe responsável exclusivamente por executar os comandos SQL (INSERT, SELECT etc.) no banco.
- **Repository pattern**: atua como um mediador entre a fonte de dados (SQLite ou API) e a lógica de negócio, decidindo de onde os dados devem ser recuperados.

Veja a seguir um exemplo de código com operação CRUD em Dart (sqlite):

```
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class TarefaDAO {
  // Inicialização e criação da estrutura de tabelas
  Future<Database> _getDatabase() async {
    final caminho = join(await getDatabasesPath(), 'app_tarefas.db');
    return openDatabase(caminho, version: 1, onCreate: (db, v) {
      return db.execute('CREATE TABLE tarefas (id INTEGER PRIMARY KEY, titulo TEXT)');
    });
  }

  // Operação de Inserção (Create)
  Future<int> inserir(Map<String, dynamic> dados) async {
    final db = await _getDatabase();

    return await db.insert('tarefas', dados, conflictAlgorithm: ConflictAlgorithm.replace);
  }

  // Operação de Consulta (Read)
  Future<List<Map<String, dynamic>>> listar() async {
    final db = await _getDatabase();

    return await db.query('tarefas', orderBy: 'id DESC');
  }
}
```


Uma das grandes vantagens da implementação do SQLite em nível superior é a viabilização do conceito offline-first. Nesta abordagem, a aplicação prioriza a leitura e escrita no banco local, sincronizando os dados com a nuvem apenas quando há conectividade estável. Isso garante que o usuário nunca tenha sua produtividade interrompida por instabilidades de rede.

Quadro 14 – SharedPreferences versus SQLite

Atributo	SharedPreferences	SQLite (relacional)
Modelo de dados	Chave-valor (NoSQL simples)	Tabelas e colunas (SQL)
Complexidade	Baixa (dados primitivos)	Alta (dados relacionais)
Desempenho	Rápido para valores isolados	Otimizado para buscas e listas
Uso profissional	Tokens, temas e flags de app	Cache de APIs, inventários e históricos



Observação

O uso do SQLite exige atenção às migrações de banco de dados. Sempre que a estrutura de uma tabela for alterada (como adição de uma nova coluna), é necessário gerenciar a versão do banco para que os dados dos usuários não sejam perdidos durante a atualização do aplicativo na loja.

6.3 Integração com recursos de hardware e sensores

A capacidade de interagir com o hardware do dispositivo é o que distingue as aplicações móveis de softwares web convencionais. O acesso a sensores como GPS (geolocalização), câmera, acelerômetros e sistemas de arquivos permite a criação de contextos de uso inteligentes e interativos. Contudo, essa integração exige que o desenvolvedor domine o ciclo de vida das permissões e o tratamento de estados assíncronos.

Modernamente, tanto o Android quanto o iOS operam sob o modelo de permissões em tempo de execução (runtime permissions). Isso significa que declarar a necessidade de um recurso nos arquivos de manifesto (AndroidManifest.xml e Info.plist) é apenas o primeiro passo; o aplicativo deve obrigatoriamente solicitar o consentimento explícito do usuário durante o uso.

```
import 'package:geolocator/geolocator.dart';

/// Gerencia a obtenção da localização com verificação de privilégios

Future<void> obterLocalizacaoAtual() async {

    bool servicoAtivo;
```

```
LocationPermission permissao;

// 1. Verifica se o hardware de GPS está habilitado

servicoAtivo = await Geolocator.isLocationServiceEnabled();

if (!servicoAtivo) return Future.error('O serviço de localização está desativado.');
```

```
// 2. Verifica o status atual da permissão

permissao = await Geolocator.checkPermission();

if (permissao == LocationPermission.denied) {

    // Solicita autorização formal ao usuário

    permissao = await Geolocator.requestPermission();

    if (permissao == LocationPermission.denied) {

        return Future.error('Permissão negada pelo usuário.');
```

```
    }

}

// 3. Acesso ao recurso de hardware após validação

Position posicao = await Geolocator.getCurrentPosition(

    desiredAccuracy: LocationAccuracy.high

);

print('Coordenadas: ${posicao.latitude}, ${posicao.longitude}');
```

```
}
```

Ao integrar recursos de hardware e de sensores, há alguns que são essenciais:

- **Geolocalização (GPS):** essencial para logística, geofencing (cercas virtuais) e serviços baseados em contexto local.
- **Captura de imagem e biometria:** utilizados para processos de Know Your Customer (KYC), autenticação segura e redes sociais.
- **Sensores de movimento:** utilizados em aplicações de saúde e acessibilidade para detectar quedas ou orientar a interface.

Para que a integração ocorra, o desenvolvedor deve realizar as configurações nativas obrigatórias:

- **Android (AndroidManifest.xml):** inclusão de tags `<uses-permission>` para declarar o uso do hardware.
- **iOS (Info.plist):** inclusão de chaves como `NSLocationWhenInUseUsageDescription`, contendo a justificativa clara do porquê o aplicativo necessita do dado. A ausência dessa justificativa resulta em rejeição automática na App Store.

Quadro 15 – Matriz de permissões e justificativas

Recurso	Declaração técnica	Justificativa de UX (exemplo)
Localização	ACCESS_FINE_LOCATION	Necessário para encontrar pontos de entrega próximos
Câmera	CAMERA	Utilizada para digitalizar códigos de barras e documentos
Arquivos	READ_EXTERNAL_STORAGE	Permite o upload de fotos de perfil da sua galeria



Observação

Segundo as diretrizes da Apple e do Google, o desenvolvedor deve aplicar o princípio do privilégio mínimo. Logo, deve-se solicitar apenas as permissões estritamente necessárias para a funcionalidade atual e explicar ao usuário o benefício imediato da concessão do acesso.

6.4 Arquitetura offline-first e governança de permissões

A maturidade de uma aplicação móvel é medida por sua capacidade de manter a operabilidade em cenários de conectividade instável e pelo respeito à privacidade do usuário. Assim, abordaremos como orquestrar a sincronização de dados e o ciclo de vida de permissões, pilares fundamentais para sistemas de nível empresarial.

O paradigma offline-first indica que a aplicação deve priorizar o armazenamento local (SQLite/Cache) e realizar a sincronização com o servidor de forma assíncrona e transparente. Isso garante consistência de informações e uma experiência de usuário sem interrupções, tendo, assim, diversas vantagens:

- **Continuidade de serviço:** o usuário pode realizar transações (como cadastros ou pedidos) mesmo sem internet.
- **Eficiência de banda:** sincronizações automáticas podem ser programadas para ocorrer apenas via Wi-Fi, reduzindo o consumo de dados móveis.
- **Performance percebida:** a leitura de dados locais é instantânea, eliminando a percepção de latência de rede.

A utilização de recursos sensíveis (como GPS, câmera, microfone etc.) é estritamente regulada pelos sistemas operacionais Android e iOS. O desenvolvedor deve implementar uma lógica de **solicitação justificada**, onde o usuário compreende o valor da concessão do acesso.

O exemplo a seguir traz a implementação de gestão de permissões com geolocalização (Dart).

```
import 'package:geolocator/geolocator.dart';

import 'package:flutter/material.dart';

class LocalizacaoService {

  /// Solicita autorização e recupera a posição geográfica

  Future<void> gerenciarPermissaoEContexto(BuildContext context) async {

    LocationPermission permission;

    // 1. Verifica status atual da permissão

    permission = await Geolocator.checkPermission();

    if (permission == LocationPermission.denied) {

      // 2. Solicitação formal de acesso (Runtime Permission)

      permission = await Geolocator.requestPermission();

      if (permission == LocationPermission.denied) {

        _notificarErro(context, 'Acesso à localização negado.');
```

```
        return;

      }

    }

    if (permission == LocationPermission.deniedForever) {

      _notificarErro(context, 'Permissão permanentemente negada. Altere nas configurações.');
```

```
      return;

    }

    // 3. Execução do recurso após validação

    Position position = await Geolocator.getCurrentPosition();

    print('Localização obtida: ${position.latitude}, ${position.longitude}');
```

```
  }
```

```
void _notificarErro(BuildContext context, String msg) {  
    ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(msg)));  
}  
}
```

Ainda, conforme as diretrizes da App Store e Google Play, a declaração de permissões deve ser binária: funcional (no código) e declarativa (nos manifestos).

- **Android (AndroidManifest.xml):** é necessário utilizar o código a seguir:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

- **iOS (Info.plist):** é obrigatório incluir a chave `NSLocationWhenInUseUsageDescription` com uma mensagem clara para o usuário. A ausência desta descrição é motivo de rejeição sumária na loja.



Observação

Segundo a OWASP Foundation (s.d.), o acesso a recursos sem necessidade clara ou sem o devido tratamento de negação pelo usuário pode ser interpretado como comportamento malicioso. Assim, sempre trate o cenário onde o usuário nega a permissão, oferecendo alternativas funcionais ou explicações adicionais de UX.

6.5 Sincronização de dados e testes de integração

A robustez de uma aplicação móvel profissional depende da sua capacidade de operar em cenários de conectividade intermitente e da validação rigorosa de seus fluxos de dados. Assim, é importante nos debruçarmos sobre a arquitetura de sincronização e como garantir sua integridade através de testes automatizados.

A sincronização é o processo de coordenação de dados entre o armazenamento local (SQLite/Cache) e o servidor remoto. Em sistemas de nível superior, utilizamos o conceito de consistência eventual, onde o aplicativo garante que, embora os dados possam divergir temporariamente devido à falta de conexão, eles serão reconciliados assim que a conectividade for restabelecida. Para tanto, é necessário um fluxo de engenharia de sincronização que segue:

- **Captura local:** a ação do usuário é persistida imediatamente no banco de dados local (por exemplo, SQLite).
- **Fila de sincronização (queue):** a operação é enviada para uma fila de processamento em segundo plano (background task).
- **Verificação de conectividade:** o sistema monitora o status da rede.

- **Reconciliação:** ao detectar internet, o app envia o payload para a API e trata possíveis conflitos de versão (por exemplo, o mesmo dado alterado em dois dispositivos diferentes).

Como afirmam Pressman e Maxim (2021), a automação de testes é fundamental para reduzir a regressão de falhas no processo de sincronização. No contexto de APIs, os testes de integração verificam se o contrato entre o frontend e o backend está sendo respeitado, validando a serialização JSON e o tratamento de códigos de status HTTP.

```
import 'package:flutter_test/flutter_test.dart';

import 'package:http/http.dart' as http;

import 'package:mockito/mockito.dart';

// Simulação (Mock) do cliente HTTP para testes controlados

class MockClient extends Mock implements http.Client {}

void main() {

  test('Deve retornar um Objeto de Usuário se a chamada HTTP for 200', () async {

    final client = MockClient();

    // Configura o Mock para retornar sucesso e um JSON válido

    when(client.get(Uri.parse('https://api.exemplo.com/user')))

      .thenAnswer((_) async => http.Response('{\"name\": \"Alex\"}', 200));

    final response = await client.get(Uri.parse('https://api.exemplo.com/user'));

    expect(response.statusCode, 200);

    expect(response.body, contains('Alex'));

  });

  test('Deve lançar exceção em caso de erro 404', () async {

    final client = MockClient();

    when(client.get(any))

      .thenAnswer((_) async => http.Response('Not Found', 404));

    final response = await client.get(Uri.parse('https://api.exemplo.com/user'));

    expect(response.statusCode, 404);

  });

}
```

A implementação de testes de integração permite validar:

- **Contratos de interface:** garante que o app não quebre se a API mudar um campo.
- **Tratamento de erros:** verifica se o app exibe a mensagem correta em caso de timeout ou erro 500.
- **Segurança:** valida se os tokens de autenticação estão sendo enviados corretamente nos cabeçalhos (headers).



Observação

Para o desenvolvedor, a sincronização deve prever a resolução de conflitos. Se um usuário edita uma nota offline e outro a edita online, o sistema deve ter uma lógica (como "o último a salvar vence" ou mesclagem de dados) para evitar a perda de informações.



Resumo

Nesta unidade, o percurso de aprendizagem avançou da camada de apresentação para a camada de infraestrutura e dados, capacitando o aluno a integrar aplicações mobile a ecossistemas digitais complexos. A compreensão de que um aplicativo moderno não é uma entidade isolada, mas um nó em uma arquitetura distribuída, é o diferencial que permite a construção de soluções dinâmicas, escaláveis e de alto desempenho.

Ao longo dos títulos, consolidaram-se competências fundamentais, como:

- **Interoperabilidade via APIs RESTful:** o domínio da comunicação cliente-servidor, utilizando o protocolo HTTP e a serialização JSON, permitiu transformar interfaces estáticas em sistemas alimentados por dados em tempo real.
- **Persistência e resiliência:** o estudo do armazenamento local (SharedPreferences e SQLite) demonstrou como implementar estratégias offline-first, garantindo que a aplicação mantenha sua funcionalidade e performance mesmo em condições de conectividade degradada.
- **Exploração de hardware e sensores:** a integração com recursos nativos, como GPS e câmera, evidenciou o potencial das plataformas móveis para oferecer soluções contextuais e interativas que transcendem o desktop.

Um marco importante nesta etapa foi a introdução à segurança e gestão de permissões. Estudamos que o acesso a dados sensíveis exige conformidade com as diretrizes das lojas (Google Play e App Store) e respeito à privacidade do usuário. O tratamento de exceções e a validação rigorosa de dados foram apresentados não apenas como tarefas técnicas, mas como práticas de garantia de qualidade (QA) essenciais para evitar falhas críticas e vulnerabilidades.

A unidade também mostrou os pilares para o desenvolvimento de aplicações móveis dinâmicas e conectadas. Os tópicos centrais incluíram:

- **Integração com APIs e JSON:** consumo de serviços externos e desserialização tipada de dados.
- **Persistência de dados:** uso técnico de SharedPreferences e SQLite para armazenamento estruturado e seguro.

- **Acesso a recursos nativos:** implementação de geolocalização e gestão de permissões em tempo de execução (runtime permissions).
- **Resiliência e sincronização:** estratégias para funcionamento offline e integridade de dados entre cliente e servidor.

Estes conhecimentos fornecem a base técnica e ética necessária para construir soluções móveis alinhadas às exigências de segurança, performance e usabilidade do mercado atual.



Exercícios

Questão 1. Um aplicativo de acompanhamento de entregas consome uma API REST para listar pedidos, atualizar status e registrar ocorrências. Em testes de campo, a equipe percebeu falhas intermitentes de conexão, respostas HTTP com códigos diferentes, erros de conversão de JSON e inconsistências entre o dado exibido na tela e o estado real da entrega. Para revisar a integração, foram propostas algumas abordagens. Assinale a alternativa que traz a abordagem mais adequada.

- A) Concentrar as chamadas HTTP diretamente na tela, pois o usuário precisa visualizar rapidamente o resultado e a proximidade entre interface e a requisição reduz a quantidade de classes.
- B) Tratar qualquer resposta diferente de sucesso como erro genérico, exibindo uma mensagem padrão, pois a distinção entre códigos HTTP interessa apenas ao servidor.
- C) Criar uma camada de serviço ou repositório para as chamadas REST, mapear os dados para DTOs ou modelos tipados, interpretar códigos de status e tratar falhas de rede, timeout e inconsistências de contrato.
- D) Converter o JSON usando mapas dinâmicos em todas as telas, pois essa abordagem é mais flexível quando a API muda e evita a criação de classes específicas.
- E) Repetir a requisição indefinidamente até obter sucesso, pois aplicações móveis lidam com conectividade instável e devem insistir até restaurar o estado correto.

Resposta correta: alternativa C.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: colocar chamadas HTTP diretamente na interface aumenta acoplamento, dificulta testes e mistura apresentação com infraestrutura. O livro-texto trabalha a necessidade de separar responsabilidades para tornar a aplicação mais sustentável.

B) Alternativa incorreta.

Justificativa: códigos HTTP possuem significado técnico relevante. Erros de autenticação, validação, indisponibilidade, ausência de recurso e falhas de servidor exigem tratamentos diferentes. Uma mensagem genérica pode esconder problemas de contrato e prejudicar a recuperação do sistema.

C) Alternativa correta.

Justificativa: a alternativa reúne os elementos centrais da integração profissional: camada de comunicação, modelos tipados, interpretação de status HTTP, tratamento de exceções, falhas de rede e inconsistências de contrato. Essa organização reduz o acoplamento e melhora a confiabilidade do aplicativo.

D) Alternativa incorreta.

Justificativa: mapas dinâmicos podem parecer flexíveis, mas aumentam o risco de erros em tempo de execução, especialmente quando há mudanças no contrato da API. Modelos tipados e DTOs tornam a conversão mais explícita e testável.

E) Alternativa incorreta.

Justificativa: repetições sem critério podem sobrecarregar rede, servidor e bateria. Estratégias de retry devem ter limite, intervalo, análise do tipo de falha e coerência com a operação realizada.

Questão 2. Um aplicativo de vistoria técnica é usado em áreas com conexão instável. O usuário coleta fotos, coordenadas de localização, observações e dados estruturados de formulários. O aplicativo precisa permitir trabalho offline, registrar permissões de acesso ao hardware e sincronizar os dados quando a conexão voltar, sem duplicar registros nem perder alterações locais. A partir disso, a equipe discute como armazenar e sincronizar as informações.

Considerando a persistência local, SQLite, recursos do dispositivo, permissões em tempo de execução e arquitetura offline-first, qual proposta é a mais adequada?

- A) Manter todos os dados apenas em memória até a conexão voltar, pois a persistência local pode criar conflitos com o servidor e dificultar a sincronização.
- B) Usar SQLite para os dados estruturados, armazenar metadados de sincronização, registrar estados pendentes, tratar permissões de câmera e localização em tempo de execução e sincronizar com controle de conflitos.
- C) Usar apenas SharedPreferences para todos os registros, pois essa solução é simples, persistente e suficiente para qualquer dado coletado em campo.
- D) Exigir conexão permanente antes de liberar o preenchimento da vistoria, pois isso impede divergências entre banco local e servidor.
- E) Solicitar todas as permissões no primeiro acesso ao aplicativo, mesmo antes de o usuário acionar câmera ou localização, pois isso reduz interrupções durante a vistoria.

Resposta correta: alternativa B.

Análise das alternativas

A) Alternativa incorreta.

Justificativa: dados mantidos apenas em memória podem ser perdidos com fechamento do aplicativo, queda de bateria ou reinicialização do sistema. Em contexto offline, a persistência local é requisito técnico, não um elemento acessório.

B) Alternativa correta.

Justificativa: a alternativa combina os pontos técnicos necessários: SQLite para dados estruturados, metadados de sincronização, controle de pendências, permissões em tempo de execução e resolução de conflitos. Essa abordagem corresponde à lógica offline-first apresentada no livro-texto.

C) Alternativa incorreta.

Justificativa: SharedPreferences é adequado para pares simples de chave e valor, como preferências e pequenas configurações. Para formulários estruturados, estados de sincronização, relações entre entidades e consultas locais, SQLite é mais apropriado.

D) Alternativa incorreta.

Justificativa: exigir conexão permanente contradiz o cenário de uso em campo e reduz a utilidade do aplicativo. Em aplicações móveis, a arquitetura deve lidar com conectividade intermitente.

E) Alternativa incorreta.

Justificativa: solicitar permissões antes do contexto de uso pode reduzir a confiança do usuário e contrariar boas práticas de privacidade. O acesso à câmera e localização deve ser justificado no momento adequado e restrito ao necessário.